RAVN DESIGN · ROUND ROBIN · ENGINEERING TRACK

# Vello NeighborCard
# Auditoría de Fidelidad de Diseño

Adriana · Septiembre 2026
Entregable del Viernes

RESUMEN

## Qué hicimos y por qué importa

Le pedimos a Claude que generara una NeighborCard para Vello desde cero — la tarjeta de vecino de la pantalla Home. La primera versión se veía razonable, pero cuando la revisamos contra el sistema de diseño, encontramos drift de tokens, errores de estructura de datos y problemas de accesibilidad.

El componente tiene: foto real del proveedor (no iniciales), marca de verificación (shield SVG), badge verde de "Disponible", bio, precio con "desde $X / unidad", distancia con ícono de pisadas en pill, rating de 5 estrellas, y un chevron de navegación.

Aquí explicamos qué salió mal, cómo mejoramos las reglas, cómo quedó el componente final, y qué preguntas aún tenemos.

EL ARTEFACTO MÁS VALIOSO

## CLAUDE.md — Las reglas que le damos a Claude para que no se desvíe

CLAUDE.md es básicamente un archivo de instrucciones que Claude lee antes de generar cualquier cosa. Es como decirle "estas son las reglas, no te las saltes."

La primera versión era muy simple: listaba 6 colores y decía "no inventes." Claude siguió las reglas... técnicamente. Usó esos colores, pero como eran primitivos y no tokens semánticos, todo estaba conectado al valor incorrecto.

Lo que aprendimos: las reglas solas no son suficientes. También necesitas el contexto completo. Reglas sin contexto = output que sigue la letra pero no el espíritu.

Fuentes que cargamos como contexto:
- Vello Design System (vello-design-system.vercel.app)
- Vello Product Brief (roles, flujos, la apuesta de confianza local)
- Vello Prototype HTML (las clases reales de la app y su estructura)
- Transcripciones de entrevistas (6 de Kestrel Park, junio 2026)

Y así fue como el guardrail mejoró con cada fallo:

`Iteration 1 — Weak guardrail`
We gave it 6 raw colors and said "dont invent." Claude used them, but since they were hex and not semantic tokens, everything was tied to the wrong value.

`Iteration 2 — Semantic tokens + data contracts`
Now we required semantic tokens: var(--surface-card), not #FFF. Fixed the data structure and pricing to support the "from $X / unit" pattern.

`Iteration 3 — Accessibility + states`
44px min on buttons, focus ring, prefers-reduced-motion. Card is <button>, not <div>. Verified mark uses native SVG, not Lucide icon.

`Iteration 4 — Integration rules`
After the floating overlay (felt outside the app), we added the rule: use Vellos native nb structure.

El guardrail terminó siendo un documento reutilizable. El siguiente componente empieza con mejores reglas.

QUÉ SALIÓ MAL EN EL OUTPUT DE CLAUDE

## Errores que encontramos revisando la primera versión

### 1. Drift de tokens + error semántico de color — tokens válidos, significado incorrecto

Claude usó colores que visualmente coincidían con Vello pero puso hex crudo en vez de tokens del sistema de diseño. El fondo de la card usaba #F6F2E7 (paper) en vez de var(--surface-card). La marca de verificación usaba un ícono Lucide en vez del shield SVG nativo de Vello.

Críticamente, cuando se agregó el badge de Disponible, usó coral/rojo (--accent, #F0623B) — un token válido pero semánticamente incorrecto. "Disponible" es un estado positivo y debe ser verde. Un punto rojo junto a "Disponible" contradice el significado. Esto se detectó durante la revisión visual del screenshot renderizado, no en la auditoría de código o tokens.

Corregido a: var(--brand-primary) para texto/punto, var(--surface-brand-tint) para fondo.

### 2. Contrato de datos — Claude asumió cosas que no son ciertas

La primera versión pasaba el rating como solo un número (4.9) sin estrellas. El prototipo de Vello usa 5 estrellas rellenas + el número. También omitió el estado "disponible" por completo.

La distancia solo soportaba "min walk" pero el prototipo tiene tres formatos: caminata, cuadras y millas. Eso es un cambio de modelo de datos, no un cambio de UI.

### 3. Accesibilidad — Funciona con mouse pero no con teclado

La card es un <button> en el prototipo pero Claude la hizo un <div> con onClick. Eso rompe la navegación por teclado. El chevron no tenía aria-hidden. Las estrellas del rating no tenían aria-label.

Estas son reglas medibles (WCAG, ARIA). Ingeniería puede verificarlas antes de publicar.

### 4. Error semántico de color — detectado en revisión visual

Después de corregir los errores iniciales, el badge de Disponible se agregó con --accent (coral, #F0623B) — un token de diseño válido. Pero "Disponible" es un estado positivo; mostrarlo en rojo contradice el significado. Ni la auditoría de código ni la de tokens lo detectaron — solo fue visible al revisar el screenshot renderizado.

Antes: punto coral + texto coral sobre fondo con tinte coral
Después: punto verde (--brand-primary) + texto verde sobre fondo --surface-brand-tint

Esto revela una brecha del guardrail: CLAUDE.md exigía usar tokens pero no tenía regla sobre la adecuación semántica del color para indicadores de estado.

### 5. Estrellas sin rellenar — detectado en revisión visual

Los íconos de las 5 estrellas del rating se renderizaron como contornos vacíos en vez de estrellas ámbar rellenas. El ícono lucide 'star' es solo trazo por defecto. Los usuarios ven estrellas huecas junto a '4.9' lo cual parece 0 estrellas — contradice visualmente el rating alto.

Corregido reemplazando los íconos outline con formas SVG path rellenas usando el token --rating (amber-500).

### 6. Chevron sin fondo circular

El ícono chevron '>' no tenía fondo en la versión generada. El diseño de referencia lo muestra dentro de un círculo gris sutil, que provee una mejor affordance de toque y un ancla visual. Se agregó un frame circular con relleno sutil alrededor del ícono.

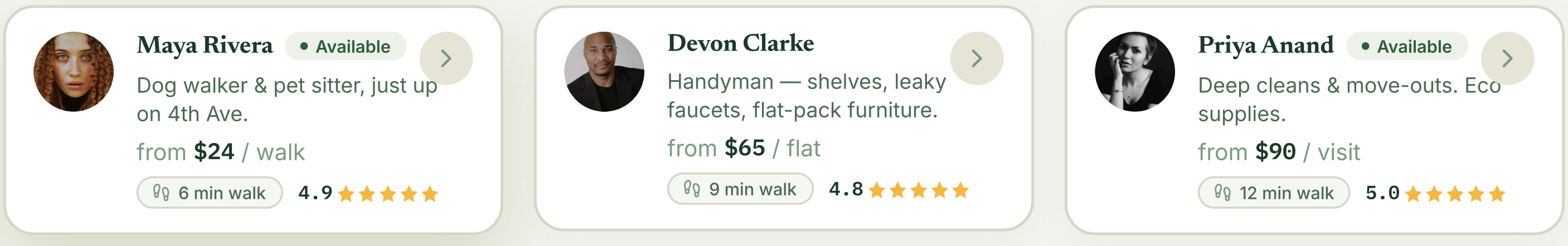
### 7. Pill de distancia sin fondo verde

El pill de distancia ('6 min walk') solo tenía un borde sin relleno. El diseño de referencia muestra un fondo verde sutil dentro del pill. Se agregó un relleno verde claro para coincidir con la referencia.

EL COMPONENTE FINAL

## Cómo quedó después de corregir los errores

Este componente no existía antes — lo generamos con Claude y lo corregimos. La versión final usa tokens semánticos, shield SVG nativo, datos estructurados con tres formatos de distancia, y soporta estados de disponibilidad.



**Maya Rivera** • Available
Dog walker & pet sitter, just up on 4th Ave.
from **$24** / walk
6 min walk · 4.9 ★★★★★

**Devon Clarke**
Handyman — shelves, leaky faucets, flat-pack furniture.
from **$65** / flat
9 min walk · 4.8 ★★★★★

**Priya Anand** • Available
Deep cleans & move-outs. Eco supplies.
from **$90** / visit
12 min walk · 5.0 ★★★★★

`State 1: Available + featured`
Verified, green Available badge, photo, bio, stars, distance pill.

`State 2: Not available`
No Available badge. User knows provider exists but isnt free right now.

`State 3: Perfect rating`
5.0 rating, all stars filled. Available badge present.

RESUMEN DE AUDITORÍA

## Qué se corrigió, específicamente

`Drift de tokens (7 propiedades)`

Superficie de card
#F6F2E7 (paper) → var(--surface-card)

Badge Disponible
coral hardcodeado → var(--brand-primary) verde + var(--surface-brand-tint)

Marca verificada
Lucide ShieldCheck → shield SVG nativo de Vello

Estrellas de rating
ámbar hardcodeado → var(--rating)

Pill de distancia
div con borde gris → pill con var(--border-default) + fondo verde

`Accesibilidad (5 correcciones)`

Elemento raíz
<div onClick> → <button type="button">

Chevron decorativo
sin aria → aria-hidden="true"

Rating accesible
sin label → aria-label="Rated 4.9 out of 5"

Foto de avatar
<img> sin alt → alt={neighbor.name}

Formatos de distancia
solo walk → walk, blocks, mi como campos separados

PREGUNTAS PARA EL DISEÑADOR

## Cosas que no pudimos resolver sin preguntar

Cuando revisamos el componente, hubo valores que no pudimos rastrear a ningún token. En vez de adivinar, los dejamos como preguntas abiertas.

1. ¿NeighborCard usa --shadow-md, --shadow-brand (featured), o sin sombra?

Por qué importa: Usamos shadow-md para normal y shadow-brand para featured, pero no encontramos la elevación oficial en el sistema de diseño.

2. El badge Disponible ahora usa --brand-primary (verde). ¿Es correcto?

Por qué importa: Originalmente usamos coral, ahora verde. Si hay un token semántico específico como --surface-status-positive, deberíamos usarlo.

3. ¿Qué pasa cuando un vecino no está disponible — ocultar la card o mostrar sin badge?

Por qué importa: Ahora mostramos la card sin el badge. El diseñador podría querer atenuarla o moverla al final.

4. La marca verificada usa var(--brand-primary) para el shield. ¿Es correcto o hay un token específico de verificación?

Por qué importa: El prototipo usa var(--green-600) directamente. Si el color de verificación cambia independientemente, necesita su propio token.

QUÉ CONTRIBUYÓ INGENIERÍA

## No solo implementar — detectar cuando una decisión de UI es una decisión de sistema

- Encontró drift invisible: Claude usó un ícono Lucide para verificación en vez del shield SVG nativo de Vello.
- Convirtió el modelo de distancia de un formato (walk) a tres campos (walk, blocks, mi).
- Descubrió que Claude omitió la disponibilidad por completo. Sin el badge, los usuarios no pueden saber si pueden reservar ahora.
- Cambió el elemento raíz de div a button — una card tocable debe ser un botón para teclado y lectores de pantalla.
- Descubrió que un componente puede renderizar perfecto y aún fallar como producto: el overlay estaba fuera del flujo de Vello.
- Detectó error semántico de color en revisión visual: el badge Disponible usaba coral/rojo (--accent) — token válido pero significado incorrecto para un estado positivo. Cambiado a verde (--brand-primary). Este error pasó las auditorías de código y tokens; solo un screenshot renderizado lo reveló.

FUENTES Y PROCESO

## De dónde vino cada decisión

Este entregable no se construyó en el vacío. Cada decisión se remonta a una fuente concreta:

`Vello Design System`
vello-design-system.vercel.app — Tokens de color (primitivos y semanticos), tipografia (Bricolage, Hanken, JetBrains Mono), escala de spacing 4px, radii, elevaciones, focus ring. Fuente para la tabla de tokens completa en CLAUDE.md.

`Vello Product Brief`
Define los tres roles (Requester, Provider, Community Admin), el scope hiperlocal, y la apuesta central: "trust scales locally." Fuente para las reglas de trust semantics y el contrato de datos en el guardrail.

`Vello Prototype (HTML)`
Archivo HTML descargable con el flujo completo: Home, request, booking, messages, profile, admin. Fuente para las clases nativas (section, neighbors, nb) y la referencia visual. La home screen es la referencia pulida; el resto tiene issues de UX plantados.

`Discovery Interviews (6 transcripts)`
Kestrel Park, junio 2026. Tres requesters, dos providers, un community admin. Evidencia para la separacion de trust signals (verificacion ≠ rating ≠ vouch) y para el pricing no-fijo.

`Claude + CLAUDE.md (el guardrail)`
Claude genero el componente inicial. CLAUDE.md actuo como el contrato del design system para Claude — tokens, trust, datos, a11y, integracion. Se itero 4 veces despues de fallos concretos.

Cada corrección se remonta a una de estas fuentes. Nada se decidió porque "se ve bien."

Lección más grande: se necesitan tres capas de revisión. (1) Auditoría de tokens — ¿los tokens son correctos? (2) Auditoría de accesibilidad — ¿los elementos son accesibles por teclado? (3) Revisión visual — ¿el output renderizado tiene sentido semántico? El badge coral de Disponible pasó las capas 1 y 2 pero falló en la 3. Un guardrail que exige tokens igual puede fallar cuando un token válido se usa en el contexto incorrecto.

PROMPTS QUE USAMOS

## Qué le pedimos a Claude en cada paso

Los prompts que usas dicen mucho sobre si entiendes el problema. Estos son los que usamos.

`Step 1 — Generate the component`
Context loaded: Vello design system, product brief, CLAUDE.md v1 (weak).

```
"Generate a NeighborCard for Vello using the design system tokens. The card shows: neighbor
photo with verified mark, name, Available badge, bio, pricing with 'from $X / unit', distance
with footprints in a pill, 5-star rating, and a navigation chevron. Its a tappable button. Use
React + Lucide icons. Follow CLAUDE.md rules."
```

Result: Rendered but with drift: raw hex, Lucide icon instead of native SVG shield, no availability, div instead of button.

`Step 2 — Audit token by token`
Context loaded: generated component, full design system, CLAUDE.md v2.

```
"Compare this NeighborCard property by property against the Vello Design System. For each visual
value, tell me: what token it should use, what the component used, whether it matches or
drifted, and the correction. Dont accept 'looks similar' — it has to be the exact token and
native pattern."
```

Found 7 properties with drift. All corrected to semantic tokens.

`Step 3 — Find product and accessibility errors`
Context loaded: corrected component, CLAUDE.md v3 with a11y rules, product brief.

```
"Review this NeighborCard against WCAG: is the card a button or div? Tap targets min 44px?
Chevron has aria-hidden? Stars have aria-label? Photo has alt text? Does Available badge rely on
color alone? Is there prefers-reduced-motion?"
```

Found: div instead of button, chevron without aria-hidden, rating without aria-label. All fixed.

`Step 4 — Integrate into Vello app`
Context loaded: Vello prototype HTML, CLAUDE.md v4 with integration rules.

```
"Integrate the NeighborCard into the Vello HTML prototype using native classes: nb, nb__avatar,
nb__vmark, nb__body, nb__top, nb__name, nb__avail, nb__bio, nb__price, nb__meta, nb__walk,
nb__tap. It should look like another neighbor in the Trusted on your block list."
```

First: overlay (failed). Second: large section (dominated). Third: native nb structure, fit in.

Patrón: cada prompt cargó más contexto. El guardrail se actualizó después de cada fallo. No hay prompt mágico — solo generar, revisar, corregir reglas, regenerar.